TECHNICAL ARCHITECTURE • DOC 1 / 3

API Architecture & Configuration

How the iCare MC patient and doctor apps discover, authenticate to, and communicate with their cloud APIs.

Subject	com.icaremc.app (patient) & com.icaremc.doctors (doctor)
Platform	Flutter (Dart AOT) on Android, shared Supabase backend
Backend	Supabase + Vercel micro-APIs (SMS / Push / Chapa banks)
Source	Static analysis of release APKs (JADX + libapp.so strings)
Status	Reverse-engineered — names/paths recovered from AOT metadata
Version	patient v1.0.1 (build 6) • doctor v1.0.0 (build 3)

Contents

1	Executive summary
2	API surface at a glance
3	Runtime configuration & the .env contract
4	The Supabase API (REST, Auth, Realtime, Storage, Edge)
5	Authentication & token lifecycle
6	Companion HTTP micro-APIs (SMS, Push, Chapa banks)
7	Chapa payment API integration
8	Firebase Cloud Messaging channel
9	Request construction, headers & error handling
10	Security observations & recommendations

1 Executive summary

iCare MC is a two-app maternal & child-health product: a **patient app** (`com.icaremc.app`, Dart package `icare_mc`) and a **doctor app** (`com.icaremc.doctors`, Dart package `icare_doctors`). Both are built with Flutter and compiled to native ARM/x86 libraries (`libapp.so`). They are thin native shells over a Dart runtime; virtually all API logic lives in Dart.

The apps do **not** talk to a bespoke monolithic backend. Instead they compose several managed and serverless APIs:

- **Supabase** is the primary API — it provides the PostgREST database API, GoTrue authentication, Realtime websockets, Storage, and Edge Functions. Both apps share the *same* Supabase project (`bjripaytpikgjvroypru.supabase.co`).
- A **Vercel-hosted admin API** (`icaremc-admin.vercel.app`) exposes push / broadcast endpoints and a public Chapa-banks lookup.
- A **Betegna AI SMS gateway** (`betegna-ai.vercel.app`) issues and verifies phone OTP codes.
- **Chapa** (Ethiopian payment processor) is called for in-app checkout/charge and validation.
- **Firebase Cloud Messaging** delivers device push notifications.

> **How these names were recovered**
>
> Release builds are obfuscated at the Java/Kotlin layer (R8/ProGuard), but Flutter's AOT snapshot retains Dart library URIs, class names, and many string literals inside `libapp.so`. Endpoint paths, table names, column lists and the shipped `.env` file are therefore directly readable, which is what this document is built from.

2 API surface at a glance

The following external services are contacted by one or both apps. Base URLs are read from the bundled environment file at launch.

Service	Base URL	Role	Used by
Supabase	`bjripaytpikgjvroypru.supabase.co`	DB / Auth / Realtime / Storage / Functions	both
Admin API (Vercel)	`icaremc-admin.vercel.app`	Push, broadcast, Chapa-bank list	both
SMS gateway	`betegna-ai.vercel.app`	OTP send / verify	both
Chapa	`api.chapa.co (via SDK)`	Payment charge / validate	patient
Firebase FCM	`fcm.googleapis.com`	Push token + delivery	both
Google Play	`play.google.com/store/apps/details`	Update-prompt version check	both

Every Supabase interaction is routed through five stable sub-paths that the client library appends to the base URL:

Sub-path	Sub-API	Purpose
`/auth/v1`	GoTrue	Sign-up, sign-in, token refresh, recovery
`/rest/v1`	PostgREST	CRUD over Postgres tables & views
`/rest/v1/rpc/`	PostgREST RPC	Server-side Postgres functions
`/realtime/v1`	Realtime	Websocket subscriptions (chat, presence)
`/storage/v1`	Storage	File upload/download (photos, documents)
`/functions/v1`	Edge Functions	Deno serverless functions

3 Runtime configuration & the .env contract

Configuration is **not** hard-coded. Each app bundles a `.env` file under `assets/flutter_assets/` and loads it at start-up with the `flutter_dotenv` package (`package:flutter_dotenv/src/dotenv.dart`). Values feed the Supabase bootstrap and the various API clients.

Patient app — shipped .env keys

```
SUPABASE_URL=https://bjripaytpikgjvroypru.supabase.co
SUPABASE_ANON_KEY=<JWT, role=anon>
SUPABASE_SERVICE_ROLE_KEY=<JWT, role=service_role>   # see warning
PUSH_API_BASE_URL=https://icaremc-admin.vercel.app/
SMS_API_BASE_URL=https://betegna-ai.vercel.app
IOS_APP_STORE_ID=your_numeric_app_store_id
PLAY_STORE_URL=.../details?id=com.icaremc.app
MOCK_UPDATE_PROMPT=false
ACCESS_TOKEN_SECRET=<JWT>
```

The doctor app ships a smaller subset (Supabase URL + anon key, SMS / push base URLs, and store URLs). The two apps deliberately point at one shared Supabase project so a doctor and a patient see the same appointments, messages and profiles.

Bootstrap sequence

1. `main.dart` → `bootstrap.dart` loads `.env` via dotenv.
2. Firebase is initialised from `firebase_options.dart`.
3. `SupabaseHelper` calls `Supabase.initialize(url, anonKey)` — log line *"***** Supabase init completed *****"*.
4. GetX dependency bindings register the API service/repository singletons (see the Backend document).
5. If URL/key are missing the app logs *"Supabase is not configured. Add SUPABASE_URL and SUPABASE_ANON_KEY to .env"* and degrades.

■ Hard-coded secrets in the shipped bundle

The patient `.env` embeds a `SUPABASE_SERVICE_ROLE_KEY` and an `ACCESS_TOKEN_SECRET`. The service-role key **bypasses all Row-Level-Security**; shipping it inside a distributable APK means anyone who unpacks the app gains full read/write access to the database.
Recommendation: rotate this key immediately, remove it from client bundles, and keep service-role usage strictly server-side (Edge Functions / the Vercel admin API).

4 The Supabase API

Supabase is consumed through the official Dart client stack, whose sub-packages are all present in the binary: `supabase`, `gotrue` (auth), `postgrest` (REST query builder), `realtime_client`, `storage_client` and `functions_client`. Version banners in the binary read *Supabase v2.12.4* / *SupabaseClient v2.10.6*.

4.1 PostgREST query builder

Data access uses the fluent builder (`SupabaseQueryBuilder`) which serialises to PostgREST HTTP calls against `/rest/v1/<table>`. Recovered `select=` strings show embedded resource expansion (Postgres foreign-key joins), e.g. for a patient's appointment list:

```
GET /rest/v1/appointments?select=
  *,doctor_profiles(first_name,last_name,specialty,phone,
     profile_photo_url),
  appointments(appointment_date,time_slot,service_name,
     status,payment_status,total_amount)
Headers: apikey: <anon>   Authorization: Bearer <user JWT>
```

4.2 RPC (stored procedures)

The `/rpc/` path is present, indicating server-side Postgres functions are invoked for transactional logic (e.g. wallet credit on cancellation, booking finalisation) rather than doing multi-step writes from the client.

4.3 Realtime

Chat and message tabs subscribe over `/realtime/v1` websockets. The binary carries the guard string *"Use channel.subscribe() before pushing events"*, confirming channel-based subscriptions in `chat_repository.dart`.

4.4 Storage buckets

Bucket	Path prefix	Contents	App
childphotos	`/object/childphotos/`	Child profile photos	patient
documents	`/object/documents/`	Doctor verification docs	both
credentials	`/object/credentials/`	Credential uploads	both
drprofile	`/object/drprofile/`	Doctor profile photos	doctor
drservices	`/object/drservices/`	Service listing images	doctor

4.5 Edge Functions

The `/functions/v1` path and the bundled `functions_client` package show Deno Edge Functions are available — the natural home for privileged operations that must not run with the anon key on-device.

5 Authentication & token lifecycle

Authentication is handled by GoTrue through `utils/auth_service.dart`. iCare uses a **phone-first** model layered on top of Supabase's email/password primitives.

5.1 The phone-to-email synthesis trick

GoTrue on this project is configured for email/password. To let users sign in with just a phone number, the client synthesises a deterministic pseudo-email. The binary reveals the template `<phone>@phone.icare.app` and the log *"AuthService: phone sign-up unavailable, using email"*. So a phone `0912345678` becomes the login identity `0912345678@phone.icare.app` with a user-chosen password.

5.2 OTP for verification & recovery

Phone ownership and password recovery are proven with SMS OTP issued by the Betegna gateway (see §6), not by GoTrue's own SMS provider. The forgot-password flow: request code → `/sms/send-otp` → user enters code → `/sms/verify-otp` → client sets a new password via GoTrue.

5.3 Tokens & sessions

- GoTrue returns a JWT **access token** and a **refresh token**; the client persists the session and auto-refreshes (the `/token` path and gotrue's refresh logic are present).
- Every authenticated REST/Storage call sends `apikey: <anon>` plus `Authorization: Bearer <user access token>`; Postgres RLS then scopes rows to `auth.uid()`.
- A **guest / demo mode** exists (`guest_demo_service.dart`) that loads sample data locally and is *not* persisted until the user signs in.
- Sign-out clears the session through `session_cleanup.dart` and hits `/logout`.

6 Companion HTTP micro-APIs

Beyond Supabase, thin REST clients wrap two Vercel-hosted services. These are plain `package:http` JSON calls built in dedicated client classes.

6.1 SMS OTP gateway (Betegna AI)

Client: `core/services/sms_otp_client.dart`. Base URL from `SMS_API_BASE_URL` (falls back to the push base URL when empty). An optional `SMS_API_KEY` bearer token is supported.

Method	Endpoint	Purpose
POST	`/sms/send-otp`	Send a one-time code to a phone number
POST	`/sms/verify-otp`	Validate the code the user typed

6.2 Admin / push API (icaremc-admin)

Clients: `push_notify_client.dart` and `chapa_banks_service.dart`. Base URL from `PUSH_API_BASE_URL`.

Method	Endpoint	Purpose	App
POST	`/api/bookings/push`	Notify on booking/appointment events	patient
POST	`/api/chat/push`	Notify the other party of a new message	both
POST	`/api/broadcast`	Broadcast/admin announcements	both
GET	`/api/public/chapa-banks`	List Chapa payout banks	doctor

The Chapa-banks client logs *“PUSH_API_BASE_URL not set, using fallback banks”*, showing a graceful offline fallback list when the endpoint is unreachable.

7 Chapa payment API integration

The patient app processes appointment payments through **Chapa** via the `chapasdk` Flutter package (bundled under `flutter_assets/packages/chapasdk`). Configuration lives in `core/config/chapa_config.dart` and the flow is orchestrated by `core/services/chapa_payment_service.dart`.

7.1 Keys & configuration

- `CHAPA_PUBLIC_KEY`, `CHAPA_SECRET_KEY` and `CHAPA_FEE_PERCENT` are referenced.
- Keys are **not** baked into the client's .env — the app fetches them from Supabase (`payment_settings` table via `payment_settings_service.dart`). If missing it shows *"Chapa is not configured. Add keys in admin payment settings."*

7.2 Checkout state machine

The SDK drives a bloc (`ChapaNativeCheckoutBloc`) with explicit states recovered from the binary:

Endpoint / state	Meaning
`POST /inline/charge`	Initiate a direct/inline charge (tx_ref, amount, currency)
`POST /inline/validate`	Poll/validate the charge result
`PaymentInitiateApiError`	charge request rejected
`PaymentValidateSuccessState`	payment confirmed
`PaymentValidateApiError / NetworkError`	validation or transport failure

7.3 Wallet & refunds

On cancellation the online amount is *credited to the patient wallet* (minus non-refundable Chapa gateway fees) rather than reversed on the card — e.g. *"Appointment cancelled. @amount was credited to your wallet."*. Wallet state is served from the `patient_wallets` table.

8 Firebase Cloud Messaging channel

Push delivery uses FCM via `firebase_messaging`. The client obtains an FCM registration token and stores it in Supabase; the binary contains *"Push: failed to save fcm_token to Supabase"*, confirming the token is persisted server-side (so the admin API can target a device).

1. App registers with FCM → receives a device token.
2. Token is upserted into Supabase (per-user `fcm_token`).
3. When an event occurs (new message / booking), the app or admin backend calls the Vercel push endpoints, which look up the target token and ask FCM (`fcm.googleapis.com`) to deliver.
4. On the device, `flutter_local_notifications` renders the alert on the *"ICare notifications"* channel.

Firebase project identifiers (patient app)

Recovered from `firebase_options.dart`: App ID `847330343170:android:4b0a00c6`…, API key `AIzaSyBoMFpkMz4`…. These are client identifiers (normal to ship) but should still be scoped with Firebase App Check and API-key restrictions.

9 Request construction, headers & errors

9.1 Standard headers

```
apikey: <SUPABASE_ANON_KEY>
Authorization: Bearer <user access token | anon key>
Content-Type: application/json
X-Supabase-Client-Platform: android
X-Supabase-Client-Platform-Version: <os version>
Prefer: return=representation      # on inserts/updates
```

The two `X-Supabase-Client-Platform*` headers are added automatically by the Dart client and are visible in the binary.

9.2 Error handling & resilience

- The `retry` package is bundled — transient network failures are retried with back-off.
- Schema drift is detected and surfaced to the user: *"Cloud database is missing tables or columns. Ask your admin to run the Supabase migrations"*.
- Typed exceptions from GoTrue (`AuthWeakPasswordException`, etc.) are mapped to friendly, localised messages in `core/errors/app_errors.dart`.
- User feedback uses toasts / EasyLoading (`show_toast_dialog.dart`, `flutter_easyloading`).

9.3 Model (de)serialisation

Rows are mapped to Dart models with named factories such as `ChildRecord.fromSupabaseRow` and `ChapaBankOption.fromJson`. Column contracts are explicit in the recovered `select=` strings, giving a precise picture of the data shape (see the Backend document).

10 Security observations & recommendations

#	Observation	Risk	Recommendation
1	service_role JWT shipped in patient .env	Critical	Rotate now; never ship service_role in a client
2	anon key + endpoints readable in APK	Expected	Enforce strict RLS on every table; treat anon key as public
3	Synthetic @phone.icare.app identities	Medium	Ensure phone uniqueness & OTP gate on password reset
4	Chapa keys fetched from DB at runtime	Low/Good	Keep secret key server-side; only expose public key to client
5	No visible certificate pinning	Medium	Add TLS pinning for Supabase & payment hosts
6	Firebase API key restrictions unknown	Low	Enable App Check & restrict the Android API key

Bottom line

The API architecture is a clean, modern “BaaS + serverless glue” design: Supabase does the heavy lifting (data, auth, realtime, storage, edge), while small Vercel functions bridge SMS, push and payment-bank concerns. The single most important fix is removing the service-role key from the client bundle and relying on Row-Level Security plus Edge Functions for anything privileged.

See companion documents: **Backend Architecture & Data Model** (2/3) and **User-Interface Architecture** (3/3).